5COSC019W · OBJECT ORIENTED PROGRAMMING

Lecture 3

Collections, Generics & the Service Layer

BSc Computer Science · Level 5 · 2 Hour Lecture

LO1 LO2 LO3

Dr Barbara Villarini

Learning Outcomes

LO1

Justify collections and the service layer as core OOP design decisions

LO2

Use Spring Boot APIs to expose a collection through four REST endpoints

LO3

Build a modular system by separating controller from service

TODAY'S SESSION

Today's Agenda

01 **Why Collections?**
One product is not a catalogue

02 **List<T> and ArrayList<T>**
The workhorse of every Java project

03 **Generics — the <T> in the box**
How the compiler keeps the list honest

04 **Map<K,V> for Fast Lookup**
Finding one product without scanning all of them

05 **The Service Layer**
Moving logic out of the controller

06 **Cohesion and Coupling**
The vocabulary your report is marked on

07 **Live Code: Four CRUD Endpoints**
ProductService wired into ProductController

SECTION 01

Why Collections?

01

One product is not a catalogue.

15 minutes

LO1

LIVE DEMO

Where We Left Off

```
// Lecture 2 — one Product, hard-coded
@RestController
public class ProductController {

    @GetMapping("/product")
    public Product getProduct() {
        return new Product(
            1L, "Laptop", 899.99);
    }
}

// GET /product  →  one object. Always.
// Always the same one.
```

Where we are

- Encapsulation works — private fields, public getters, Jackson serialises it
- Swagger UI shows the endpoint and returns clean JSON
- But the shop sells exactly one product, and it can never change
- Everything today exists to fix that one sentence

LIVE DEMO

The Obvious Fix Does Not Scale

```
// "We need three products now"
Product p1 = new Product(1L, "Laptop",  899.99);
Product p2 = new Product(2L, "Mouse",    24.99);
Product p3 = new Product(3L, "Monitor", 249.99);

// ...and the controller?
@GetMapping("/products")
public Product[] all() {
    return new Product[]{ p1, p2, p3 };
}

// Now add a fourth. And a fifth.
```

✗ **Every product needs a name**

p1, p2, p3 — you must invent a variable for each one, in advance.

✗ **The count is frozen in code**

Adding a product means editing and recompiling the application.

✗ **Nothing can be added at runtime**

A customer cannot create a product. Neither can POST /products.

ASK THE ROOM

The Change Request

"The catalogue goes live on Friday with `500 products`, and the buyer adds new ones daily."

How many variables do we declare? Shout a number.

500

named variables under the p1, p2, p3 approach

365

recompiles a year, because the buyer adds one every morning

1

variable, once we use a List — and it grows by itself

THE PROMISE: One variable, sized at runtime, that the buyer can grow without a developer. That is what a collection is.

Array or List?

Both hold many values under one name. Only one of them can grow.

`Product[]` **Fixed at creation**

- new Product[500] — you must know 500 when you write the line
- No add() and no remove(). The length never changes
- Growing means creating a bigger array and copying everything across
- Fine for fixed data; wrong for a catalogue

`List<Product>` **Grows on demand**

- Starts empty; add() as many as you like, whenever you like
- remove() shrinks it again — the size is always size()
- The resizing and copying happens for you, inside the library
- This is what every real Spring Boot project uses

KEY TAKEAWAY: An array is a fixed-length box. A collection is a box that resizes itself. Almost every OOP project wants the second one.

SECTION 02

List<T> and ArrayList<T>

02

The workhorse of every Java project.

20 minutes | LO1 | LO2

One Is a Contract, One Is a Class

```
List<Product> products = new ArrayList<>();
```

`List` The interface

WHAT YOU CAN DO

- A promise: this thing is ordered, and it can add, get and remove
- It has no code of its own — you can never write new List<>()
- Declare your variables as List so the implementation can change

`ArrayList` The implementation

HOW IT ACTUALLY WORKS

- Real code, backed by an array that it resizes for you
- Fast to read by index; the default choice unless you know better
- LinkedList is another implementation of the same List contract

LECTURE 5 AND 7 PREVIEW: Programming to the interface is the same idea we will use for polymorphism and dependency injection. Notice it now; we return to it twice.

LIVE CODE

Type This Now — Your First ArrayList

```
import java.util.ArrayList;
import java.util.List;

List<Product> products = new ArrayList<>();

products.add(new Product(1L, "Laptop", 899.99));
products.add(new Product(2L, "Mouse",   24.99));

products.size();       // 2
products.get(0);       // the Laptop
products.remove(1);    // the Mouse is gone
products.size();       // 1

// no p1, no p2, no fixed length
```

What to notice

- The list starts empty — nothing says how big it will get
- The diamond <> on the right is empty; Java infers Product
- get(0) is the first element — lists count from zero
- IntelliJ will add the two imports for you with Alt+Enter

The Six Operations You Will Actually Use

`add(item)`

Appends to the end. Returns true. This is how POST /products will work.

`get(index)`

Returns the element at that position. Zero-based, so get(0) is the first.

`size()`

How many elements right now. Not a fixed capacity — it changes as you add.

`remove(index)`

Deletes and shifts everything after it left. Also remove(object).

`contains(item)`

True if the list holds an equal element. Uses equals(), not ==.

`isEmpty()`

Cleaner than size() == 0, and it says what you mean out loud.

PRACTICAL RULE: These six cover every list operation in your coursework. Reach for the documentation only when none of them fits.

LIVE CODE

Two Ways to Walk the List

```java
// 1 — the enhanced for loop. Use this.
for (Product p : products) {
    System.out.println(p.getName());
}

// 2 — forEach with a lambda
products.forEach(p ->
    System.out.println(p.getName()));

// finding one, the long way
for (Product p : products) {
    if (p.getId().equals(id)) return p;
}
return null;   // we will fix this in Lecture 6
```

What to notice

- Read the colon as “for each Product p in products”
- The lambda version does the same job in one line
- p -> … is a function you pass as a value; the full story is Lecture 8
- Searching by hand means touching every element — remember that for Section 04

QUICK CHECK

Quick Check

Q1 **Why do we write List on the left and ArrayList on the right?**

→ The variable depends only on the contract. Swap in a different implementation later and no other line changes.

Q2 **A list has 3 elements. What does get(3) return?**

→ Nothing — it throws IndexOutOfBoundsException. Valid indices are 0, 1 and 2.

Q3 **Can an ArrayList hold both a Product and a String?**

→ Not if it is declared List<Product>. The compiler stops you. Why that is a feature is Section 03.

SECTION 03

Generics — the <T> in the Box

How the compiler keeps the list honest.

12 minutes

LO1

The Same List, With and Without <Product>

WITHOUT

```
// a raw list — anything goes
List products = new ArrayList();

products.add(new Product(...));
products.add("oops");   // compiles!

Product p =
    (Product) products.get(1);
// ClassCastException at runtime
```

WITH

```
// a typed list — one kind only
List<Product> products =
        new ArrayList<>();

products.add(new Product(...));
products.add("oops");   // will not compile

Product p = products.get(1);
// no cast, and no surprises
```

The analogy: a List<Product> is a labelled box. Only Products go in, so you always know what comes out — and you never have to unwrap and check.

ASK THE ROOM

Will It Compile?

```
List<Product> items = new ArrayList<>();

// A
items.add(new Product(1L, "Mug", 8.50));

// B
items.add("Mug");

// C
String s = items.get(0);

// D
int total = items.size();
```

Vote on each line

- A compiles — a Product is exactly what the box is labelled for
- B fails — a String is not a Product, and the compiler says so
- C fails — get(0) hands back a Product, which will not fit in a String
- D compiles — size() returns an int no matter what the list holds
- Three of these are caught before the app ever runs. That is the win

What the Angle Brackets Buy You

`type safety`

Caught by the compiler

The wrong type is a red underline in the IDE, not a 500 error in production.

`no casting`

Out comes a Product

get() already returns the right type, so there is no (Product) cast to write or forget.

`readable intent`

The declaration documents itself

List<Product> tells the next developer what is inside without opening a single method.

`autocomplete`

The IDE can help you

IntelliJ offers getName() on the loop variable because it knows the element type.

IN YOUR REPORT: “The catalogue is held in a List<Product>, so the compiler guarantees every element is a Product.” That sentence is LO1 evidence.

SECTION 04

Map<K,V> for Fast Lookup

04

Finding one product without scanning all of them.

15 minutes | LO1 | LO3

LIVE DEMO

Finding One Product in Five Hundred

```
// GET /products/499 with a List
public Product findById(Long id) {
    for (Product p : products) {
        if (p.getId().equals(id)) {
            return p;
        }
    }
    return null;
}

// worst case: 500 comparisons
// for one product
```

Why this hurts

- The work grows with the catalogue — this is O(n), linear search
- Double the products and you double the comparisons
- Every GET /products/{id} pays that cost, on every request
- A Map answers the same question in one step, whatever the size

List or Map — Which One and Why

`List<Product>` **Order matters**

USE IT WHEN

- You need every item — GET /products returns the whole catalogue
- The sequence is meaningful, or duplicates are allowed
- Lookup by id costs a scan of the whole list
- Perfect for the JSON array your endpoint returns

`Map<Long,Product>` **Lookup matters**

USE IT WHEN

- You look things up by a key — the product id, a username, a code
- Keys are unique; putting the same key twice replaces the value
- get(id) goes straight there, however many entries exist
- Order is not guaranteed in a HashMap — do not rely on it

THE HONEST ANSWER: Real systems use both. Your seminar uses a List today because the API returns collections; Lecture 7 swaps in a Map behind a repository interface.

LIVE CODE

Type This Now — HashMap in Action

```
Map<Long, Product> catalogue = new HashMap<>();

// key            value
catalogue.put(1L, new Product(1L, "Laptop", 899.99));
catalogue.put(2L, new Product(2L, "Mouse",   24.99));

catalogue.get(2L);            // the Mouse, in one step
catalogue.containsKey(7L);    // false
catalogue.remove(1L);         // the Laptop is gone
catalogue.size();             // 1

// the values, as a collection
List<Product> all =
        new ArrayList<>(catalogue.values());
```

What to notice

- Two type parameters now — the key type, then the value type
- put() adds or replaces; there is no add() on a Map
- get() returns null when the key is missing. Lecture 6 fixes that properly
- values() gets you back to a collection when the endpoint needs one

ACTIVITY

You Choose — List or Map?

Hands up for each one. Reveal the verdict only after the vote.

All products, in order added

LIST

The endpoint returns the whole catalogue as an array.

Look up a user by email

MAP

The email is a unique key. Straight to the value.

Items in a shopping basket

LIST

Order matters, and the same item may appear twice.

Country code to country name

MAP

"GB" → "United Kingdom". A textbook key-value pair.

Comments under a blog post

LIST

Chronological, and duplicates are perfectly legal.

Product id to Product object

MAP

Exactly what your Lecture 7 repository will hold.

ASK THE ROOM: For each MAP, follow up with “what is the key, and can two rows ever share it?” That question is the whole test.

SECTION 05

The Service Layer

Moving the logic out of the controller.

18 minutes | LO2 | LO3

05

LIVE DEMO

Everything in the Controller

```
@RestController
public class ProductController {

    private List<Product> products
            = new ArrayList<>();
    private Long nextId = 1L;

    @PostMapping("/products")
    public Product add(@RequestBody Product p) {
        p.setId(nextId++);      // business rule
        products.add(p);        // data storage
        return p;               // HTTP response
    }
}
```

Three jobs, one class

HTTP handling, id generation and storage all live in the same file.

Untestable without HTTP

To test the id rule you must start Spring and fire a real request.

Nothing can reuse it

A scheduled job or a second controller cannot get at that list.

Two Jobs, Two Homes

C ProductController

THE WEB LAYER

- Receives HTTP requests and returns HTTP responses
- Reads path variables and request bodies; nothing more
- Knows about URLs and status codes — and no business rules at all

S ProductService

THE BUSINESS LAYER

- Owns the list and the rules — assigning ids, finding, deleting
- Knows nothing about HTTP; it would work in a desktop app unchanged
- Can be tested on its own in milliseconds, with no server running

THE TEST: If you deleted every annotation, would the class still make sense? A service says yes. A controller says no. That is how you know where code belongs.

LIVE CODE

Type This Now — Extract the Service

```java
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private List<Product> products = new ArrayList<>();
    private Long nextId = 1L;

    public List<Product> getAllProducts() {
        return products;
    }

    public Product addProduct(Product product) {
        product.setId(nextId++);
        products.add(product);
        return product;
    }
}
```

What to notice

- Not one HTTP word in the file — no @GetMapping, no ResponseEntity
- @Service tells Spring to create one and manage it for you
- The list is private: nobody outside can reach in and change it
- The id rule now lives in exactly one place

The Journey of One Request

What actually happens when the browser sends POST /products

1

The request arrives at the controller

Spring matches the URL and the HTTP method, and turns the JSON body into a Product.

2

The controller delegates immediately

One line: productService.addProduct(p). It does not decide anything itself.

3

The service applies the business rules

It assigns the next id and stores the product in its private list.

4

The service returns the saved object

Plain Java all the way back. The service has no idea a browser is involved.

5

The controller shapes the response

It hands the object to Spring, which serialises it to JSON and sets the status code.

RULE: Requests travel downwards and answers travel back up. The service never calls the controller — not once, not ever.

@Autowired — Just Enough for Today

```
@RestController
public class ProductController {

    @Autowired
    private ProductService productService;

    // no  new ProductService()  anywhere
}
```

What it does

- Spring creates one ProductService when the app starts
- It hands that same instance to whoever asks for it
- @Service is what makes it available to be handed over

What you need this week

- Annotate the service with @Service
- Annotate the controller field with @Autowired. It works

What we are NOT explaining yet

- How the container finds and builds the bean
- Why constructor injection is better — that is Lecture 7

SECTION 06

Cohesion and Coupling

06

The vocabulary your report is marked on.

12 minutes

LO3

Two Words Worth Marks

High Cohesion

ONE CLASS, ONE JOB

- Everything in the class serves the same purpose
- ProductService manages products — and does nothing else
- Test: can you describe the class in one sentence with no “and”?

Low Coupling

FEW DEPENDENCIES, ONE DIRECTION

- Classes lean on each other as little as possible
- Controller → Service. The service never looks back up
- Test: change one class — how many other files must you open?

IN YOUR REPORT: “The controller depends on the service, but not the reverse — a one-way dependency that keeps coupling low.” That is LO3 language.

ASK THE ROOM

Which One Is Easier to Test?

TANGLED

```
// to test the id rule you must:
@SpringBootTest
class ControllerTest {
  // start the whole application
  // fake an HTTP request
  // parse a JSON response
  // ...to check one ++
}
```

SEPARATED

```
// to test the id rule you:
class ProductServiceTest {
  @Test void assignsIds() {
    var svc = new ProductService();
    var p = svc.addProduct(
        new Product(null,"Mug",8.5));
    assertEquals(1L, p.getId());
  }
}
```

Rule of thumb: if a class is hard to test, it is usually doing too much or depending on too much. Difficulty writing a test is a design warning, not a testing problem.

ACTIVITY

Where Does This Code Belong?

The brief

A colleague sends you five lines of code from a Spring Boot project and asks where each one should live: in the Controller, or in the Service? Decide as a pair, and be ready to justify one of them out loud.

1. Reject a product whose price is below zero
2. Read the id out of the URL with @PathVariable
3. Give a new product the next available id
4. Return 404 when nothing matches the id
5. Filter the catalogue down to products under £50

6 minutes

then two pairs defend a choice

The verdicts

- Service — it is a business rule
- Controller — that is pure HTTP
- Service — the rule owns the ids
- Controller — 404 is a status code
- Service — filtering is logic

QUICK CHECK

Quick Check

Q1 **Your ProductService also sends confirmation emails. Which principle is broken?**

→ Cohesion. That is two jobs in one class. The email work belongs in its own service.

Q2 **Why should ProductService never hold a reference to ProductController?**

→ It would make the dependency circular. Requests go one way only: controller to service.

Q3 **Where does the ArrayList live after today's refactor, and why?**

→ Private, inside ProductService. The controller does not know or care how products are stored — that is encapsulation at the class level.

SECTION 07

Live Code: Four Endpoints

07

One service, one controller, a working API.

20 minutes | LO2 | LO3

LIVE CODE

Key Code Example — ProductService

```java
@Service
public class ProductService {

  private List<Product> products
          = new ArrayList<>();
  private Long nextId = 1L;

  public List<Product> getAllProducts() {
      return products;
  }

  public Optional<Product>
          getProductById(Long id) {
      return products.stream()
          .filter(p ->
              p.getId().equals(id))
          .findFirst();
  }
```

```java
  // ...ProductService (continued)

  public Product addProduct(Product p) {
      p.setId(nextId++);
      products.add(p);
      return p;
  }

  public void deleteProduct(Long id) {
      products.removeIf(p ->
          p.getId().equals(id));
  }
}

// Four methods. No HTTP anywhere.
// This class would work unchanged
// in a desktop app.
```

LIVE CODE

Four Endpoints, Four One-Line Methods

```java
@RestController
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping("/products")
    public List<Product> all() {
        return service.getAllProducts();
    }

    @GetMapping("/products/{id}")
    public Product one(@PathVariable Long id) {
        return service.getProductById(id)
                      .orElse(null);   // Lecture 6
    }
    // POST and DELETE next →
}
```

Why this is better

- Every method is one line: take the request, delegate, return
- No list, no nextId, no rules — the controller stores nothing
- @PathVariable pulls the id straight out of the URL
- orElse(null) is a placeholder we replace with a real 404 in Lecture 6

LIVE CODE

Searching With a Stream

```
// in ProductService
public List<Product> findByName(String name) {
    return products.stream()
        .filter(p -> p.getName()
                     .contains(name))
        .toList();
}

// the same thing, the long way
List<Product> found = new ArrayList<>();
for (Product p : products) {
    if (p.getName().contains(name)) {
        found.add(p);
    }
}
return found;
```

Read it as a sentence

- stream() — “go through the list one at a time”
- filter(…) — “keep only the ones that match”
- toList() — “collect what survived into a new list”
- Both versions do the same work; the top one says it in four lines
- Use it as a recipe this week. Lambdas properly in Lecture 8

LIVE CODE

Test It — Swagger UI Walkthrough

1

Restart the application

The new @Service class has to be picked up by the classpath scan at startup.

2

Open /swagger-ui.html

Four operations should now appear under the controller instead of one.

3

POST /products twice

Send a body with a name and a price but no id. Watch the service assign 1 and then 2.

4

GET /products

The array holds both objects. Nothing was hard-coded and nothing was recompiled.

5

GET /products/2, then DELETE it

Fetch by id, delete it, then GET the collection again and confirm it has gone.

IF IT BREAKS: A null service almost always means a missing @Service on the class, or the service sitting outside the application's package.

LECTURE RECAP

What We Covered Today

✓

Why Collections

One name, many values, sized at runtime

✓

List and ArrayList

The contract on the left, the class on the right

✓

Generics

The compiler guards what goes in the box

✓

Map<K,V>

Straight to the value, whatever the size

✓

The Service Layer

Controller handles HTTP, service owns the rules

✓

Cohesion & Coupling

One job each, one direction of dependency

✓

Live Code

Four endpoints on a service you can test

UP NEXT

Your Tutorial 3 Tasks — in the next tutorial session

1. Add a List<Product> to ProductService with the four methods from the live code
2. Wire ProductService into ProductController with @Autowired
3. Build the four endpoints: GET /products, GET /products/{id}, POST, DELETE
4. Test each endpoint in Postman and Swagger UI — check status codes, not just bodies
5. Add findByName(String name) using .stream().filter(...).toList()

```
Commit:  "Tutorial 3: ProductService with List — four CRUD endpoints working"
```

Resources

- Oracle Docs: The Collections Framework
- Baeldung: Guide to Java ArrayList
- Baeldung: Guide to Java HashMap
- Spring Docs: @Service annotation

Lecture 4 Preview

Inheritance — one Product class becomes a family, and the duplication disappears.